

Lecture 1: Deep Foundations of Continuous Integration and Continuous Delivery in MLOps

1 Introduction

Machine learning systems are not isolated predictive models; they are dynamic software systems embedded in production environments. They interact with users, data pipelines, regulatory constraints, and business objectives. Continuous Integration (CI) and Continuous Delivery/Deployment (CD) emerged in software engineering to manage system complexity and reduce operational risk [1]. However, ML systems amplify traditional software risks because behavior depends on data, stochastic optimization, and shifting environments [2]. This lecture builds the engineering foundation for CI/CD and explains why ML systems require an expanded automation framework.

2 Foundations of Continuous Integration

2.1 The Integration Problem and Its Real-World Consequences

When multiple developers work independently and integrate infrequently, integration problems accumulate.

Broken Builds: A broken build occurs when integrated code fails to compile or execute. *Real-world implication:* If a production deployment pipeline is blocked by a broken build, hotfixes cannot be released. For example, during high-traffic retail events, delayed deployment can cause revenue loss and customer dissatisfaction.

Regressions: A regression occurs when new changes unintentionally break previously working functionality. *Real-world implication:* A regression in a payment processing system may prevent successful transactions, directly affecting revenue and customer trust.

Undetected Incompatibilities: Two independently developed modules may function individually but fail when combined. *Real-world implication:* In distributed microservices systems, API version mismatches can cause cascading failures across services.

Continuous Integration enforces:

- Frequent commits to shared repositories
- Automated builds
- Automated unit and integration tests
- Immediate feedback loops

The goal is to detect failures early, when fixes are inexpensive.

2.2 Why Defect Cost Increases Exponentially When Detected Late

The cost of fixing defects grows exponentially over time because:

1. The defect propagates into dependent modules.
2. More system components rely on faulty assumptions.
3. Reproducing the failure becomes harder.
4. Customer-facing damage may occur.

Example:

A simple data preprocessing bug is introduced during development.

If detected during unit testing:

- Fix requires minutes.

If detected after deployment:

- Incorrect predictions affect customers.
- Regulatory exposure increases.
- Incident response is triggered.
- Rollback procedures must be executed.

The organizational cost becomes orders of magnitude larger. This principle is foundational to CI philosophy [1].

3 Continuous Delivery and Continuous Deployment

3.1 Continuous Delivery

Continuous Delivery (CD) ensures that every commit to the main branch produces a *production-ready artifact*. This does not necessarily mean the artifact is immediately deployed; rather, it means that it **could be deployed at any moment** without additional manual steps.

What Does “Production-Ready Artifact” Mean?

A production-ready artifact is:

- Fully built and compiled
- Tested (unit, integration, validation tests passed)
- Packaged with locked dependencies
- Versioned and traceable
- Deployable to the production environment without modification

In modern systems, this artifact is often a container image (e.g., Docker image) or a versioned model artifact stored in a registry. This eliminates the historically dangerous step of “last-minute fixes before deployment.”

Reproducible Builds

A reproducible build means that:

$$\textit{Same Code} + \textit{Same Dependencies} + \textit{Same Configuration} \Rightarrow \textit{Identical Artifact}$$

Reproducibility is critical because:

1. Debugging requires reproducing production failures.
2. Regulatory audits may require demonstrating exactly what code was deployed.
3. Incident response depends on rollback to a known stable state.

Example (Finance):

A credit scoring model produces incorrect risk scores in production. Regulators request the exact model version used during a specific week. Without reproducible builds:

- The organization cannot recreate the model.
- Compliance exposure increases.

With Continuous Delivery:

- The exact model artifact is stored and versioned.
- The environment is locked.
- The failure can be reproduced and analyzed.

This traceability is mandatory in regulated sectors.

Controlled Release Cycles

Continuous Delivery separates:

- **Build frequency** (continuous)
- **Deployment frequency** (controlled)

This allows organizations to:

- Batch releases strategically
- Schedule releases during low-risk windows
- Align releases with compliance approvals

Example (Healthcare):

A hospital deploys a diagnostic assistance model. Deployment requires:

- Clinical validation
- Legal approval

- Regulatory sign-off

Continuous Delivery allows engineers to continuously improve and validate the system, while final deployment occurs only after formal review. Thus, engineering velocity does not conflict with governance requirements.

Reduced Deployment Risk

Deployment risk is reduced because:

1. Changes are small and incremental.
2. Each change has already passed automated tests.
3. Artifacts are immutable (no last-minute modifications).

Large, infrequent deployments are risky because:

- Many changes are bundled together.
- Root cause analysis becomes difficult.
- Rollbacks are complex.

Continuous Delivery encourages small, validated increments.

Example (E-commerce):

An online retailer deploys pricing updates. Without Continuous Delivery:

- Hundreds of changes accumulate.
- A pricing bug affects thousands of transactions.

With Continuous Delivery:

- Each change is isolated.
- If an issue arises, rollback is immediate.

Smaller changes reduce systemic fragility.

When Is Continuous Delivery Especially Useful?

1. Regulated Industries (Finance, Healthcare, Insurance) In these industries:

- Audit trails are legally required.
- Model decisions may affect human welfare.
- Deployment must be explainable and reproducible.

Continuous Delivery provides:

- Version traceability
- Artifact immutability
- Documented validation pipelines

This enables compliance with regulatory frameworks (e.g., model governance policies in financial institutions).

2. High-Risk Domains Requiring Human Approval Examples include:

- Autonomous driving software
- Medical imaging diagnosis systems
- Air traffic control systems

In these domains:

- Deployment errors may cost lives.
- Human review is mandatory.

Continuous Delivery allows:

- Automated validation pipelines
- Formal approval gates
- Deployment only after expert sign-off

Thus, engineering rigor and human oversight coexist.

Summary of Continuous Delivery

Continuous Delivery provides:

- Reproducibility (for debugging and compliance)
- Controlled governance (for regulated deployment)
- Reduced systemic risk (through small validated changes)
- Operational stability (through immutable artifacts)

It is particularly valuable when deployment carries legal, ethical, or safety implications.

3.2 Continuous Deployment

Continuous Deployment is the practice of automatically releasing every validated change to production without requiring manual approval. Unlike Continuous Delivery, which ensures deployability, Continuous Deployment eliminates the human gate between validation and release. Once the CI/CD pipeline passes all tests, the system is updated in production automatically.

What Makes Continuous Deployment Possible?

Continuous Deployment requires:

- Extremely high test coverage
- Reliable automated validation
- Strong monitoring and alerting systems
- Fast rollback mechanisms

Because there is no manual review stage, confidence must come from automation and observability.

Why Continuous Deployment Reduces Risk

At first glance, automatic deployment appears riskier. However, in practice, it often reduces systemic fragility. This is because:

1. Changes are small and incremental.
2. Failures are isolated to recent commits.
3. Root Cause Analysis (RCA) becomes simpler.
4. Rollbacks are faster.

Large, infrequent releases bundle many changes together, making it difficult to identify the source of failure. Continuous Deployment minimizes the “blast radius” of each change.

Engineering Principle: System fragility increases with batch size. If N changes are deployed simultaneously, the interaction complexity grows approximately combinatorially. Smaller deployment increments reduce interaction risk.

Real-World Example: Etsy

Etsy adopted Continuous Deployment and moved to dozens of deployments per day [6].

Before automation:

- Deployments were infrequent.
- Releases were large.
- Failures were difficult to debug.

After Continuous Deployment:

- Each release contained minimal changes.
- Monitoring detected anomalies quickly.
- Rollbacks were immediate.

The insight was not “deploy faster,” but rather “deploy smaller.”

When Is Continuous Deployment Useful?

Continuous Deployment is most effective in environments where:

1. Large-Scale Web Platforms Examples include:

- Search engines
- Social media platforms
- E-commerce websites

These systems:

- Operate at massive scale.
- Have extensive monitoring infrastructure.
- Can detect performance regressions quickly.

For such systems, rapid iteration improves competitiveness and user experience.

2. A/B Testing Environments Companies frequently deploy experimental features to subsets of users.

Continuous Deployment enables:

- Rapid experimentation
- Statistical evaluation of variants
- Immediate rollback if metrics degrade

In ML systems, this allows:

- Online model comparison
- Controlled exposure of new models
- Real-time performance tracking

3. Rapid Iteration Product Teams Startups and high-growth teams benefit from:

- Fast feedback cycles
- Reduced coordination overhead
- Immediate user validation

Continuous Deployment shortens the loop between development and real-world feedback.

Continuous Deployment in ML Systems

In MLOps, Continuous Deployment may apply to:

- Model serving APIs
- Feature engineering services
- Monitoring dashboards

However, it is often applied more cautiously for:

- High-stakes decision systems
- Safety-critical models

- Regulated domains

Many organizations adopt a hybrid strategy:

- Continuous Delivery for model artifacts
- Continuous Deployment for low-risk infrastructure changes

Risk Mitigation Mechanisms

To safely implement Continuous Deployment, organizations rely on:

Canary Releases: Deploy new versions to a small subset of users first.

Feature Flags: Enable or disable features dynamically without redeployment.

Blue-Green Deployments: Maintain two production environments and switch traffic gradually.

Real-Time Monitoring: Automatically detect metric degradation and trigger rollback.

When Continuous Deployment Is Not Appropriate

Continuous Deployment may be unsuitable when:

- Regulatory approval is required before deployment.
- Model decisions have life-or-death implications.
- Monitoring infrastructure is immature.

In such cases, Continuous Delivery with manual promotion gates provides a safer balance.

Summary of Continuous Deployment

Continuous Deployment:

- Automates release to production.
- Reduces fragility through small incremental changes.
- Requires strong observability and rollback systems.
- Accelerates experimentation and feedback loops.

Its effectiveness depends on organizational maturity, testing rigor, and monitoring capability.

4 ML Systems Change the Equation

Traditional software:

$$Output = f(Code)$$

ML systems:

$$Output = f(Code, Data, Hyperparameters)$$

4.1 Non-Determinism in ML

Random Initialization: Model weights begin randomly; small changes alter final models.

Stochastic Optimization: Mini-batch gradient descent introduces randomness in update order.

Hardware Non-Determinism: GPU parallelization may produce slight numerical variation.

Implication: Two identical training runs may not produce identical results. Therefore, validation must tolerate statistical variance.

4.2 Hidden Technical Debt in ML Systems

Sculley et al. [2] identify:

Entangled Features: Features influence each other in non-transparent ways.

Undocumented Assumptions: Data preprocessing may assume stable distributions.

Feedback Loops: Model predictions influence future training data.

Example: A recommendation system promotes popular items. Over time, popular items dominate training data, reinforcing bias.

5 Drift in Machine Learning Systems

5.1 Types of Drift Explained

Covariate Drift ($P(X)$ changes): Customer demographics shift during economic changes. Example: Pandemic alters purchasing behavior.

Label Drift ($P(Y)$ changes): Fraud rate increases during economic downturn.

Concept Drift ($P(Y|X)$ changes): Spammers change tactics, breaking email spam models.

Drift requires monitoring and retraining [3].

5.2 Case Study: Google Flu Trends

Google Flu Trends predicted flu prevalence using search queries. Initial success occurred because search patterns correlated with flu outbreaks. Missteps:

- Model overfit historical correlations.
- Search behavior changed over time.

- Media coverage influenced search patterns.
- No recalibration pipeline existed.

Result: Significant overestimation of flu cases [5].

6 Fairness and Bias in ML Systems

6.1 Case Study: COMPAS

COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) predicted recidivism risk in criminal justice settings. ProPublica found that Black defendants were more likely to receive false positive risk assessments. Missteps:

- Fairness trade-offs not clearly defined.
- No transparent reporting of group-level error rates.
- Optimization prioritized predictive accuracy without fairness auditing.

Kleinberg et al. [4] showed fairness criteria cannot be simultaneously satisfied.

6.2 Case Study: Amazon Hiring Model

Amazon trained a hiring model on historical resumes. Historical data reflected male-dominated hiring patterns. Missteps:

- Training data encoded gender imbalance.
- Proxy variables (e.g., “women’s chess club”) acted as bias signals.
- Insufficient fairness auditing before deployment.

The model penalized resumes containing female-associated terms and was ultimately abandoned.

7 Conclusion

CI/CD reduces integration risk in software systems.

ML systems introduce additional risks:

- Statistical uncertainty
- Data dependency
- Drift
- Fairness trade-offs
- Reproducibility challenges

Real-world failures demonstrate that ML systems fail not only because of code defects, but because of unvalidated assumptions about data and society. Lecture 2 will transition from theory to GitHub Actions implementation.

8 In-Class Critical Thinking Exercise (10 Minutes)

Scenario

You are the ML engineering team at a financial institution deploying a credit approval model.

The model:

- Achieves 94% accuracy.
- Shows a 6% lower approval rate for a protected demographic group.
- Was trained on five years of historical loan data.
- Is deployed via CI/CD with automatic retraining every month.

Task (5 minutes small group discussion)

Answer the following:

1. Is 94% accuracy sufficient justification for deployment?
2. What additional validation would you require before promotion?
3. What fairness metrics should be examined?
4. Should retraining be automatic, or require human oversight?
5. What drift indicators would you monitor post-deployment?

Submit your answers on discord.

References

- [1] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.
- [2] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," NeurIPS, 2015.
- [3] J. Gama et al., "A Survey on Concept Drift Adaptation," ACM Computing Surveys, 2014.
- [4] J. Kleinberg et al., "Inherent Trade-Offs in the Fair Determination of Risk Scores," ITCS, 2017.
- [5] D. Lazer et al., "The Parable of Google Flu," Science, 2014.
- [6] J. Allspaw, "The Culture of Continuous Deployment," Velocity Conference, 2012.